Antigravity

AI 에이전트와 함께하는 차세대 개발 방식

우리는 지금 어디에 있는가?

이제 AI는 당신의 손가락이 아닌, '주니어 개발자'

[3세대] Antigravity & AI Agents
목표 달성을 위한 자율 계획 및 행동

[2세대] GitHub Copilot / Cursor AI
코드 자동 완성 및 채팅 보조

[1세대] 구글링 & Stack Overflow
직접 검색 및 복사

Antigravity

Antigravity란 무엇인가?

개발자의 생산성을 위한 차세대 AI 코딩 어시스턴트

PROJECT STRUCTURE ANALYSIS

DEPENDENCY MAPPING

Context-Aware

프로젝트 전체 구조와
라이브러리 이해

GLOBAL CODEBASE SUPPORT

CROSS-PLATFORM INTEGRATION

Multi-Language

C, Python 등
수십 개 언어 지원

LOGIC DECODING

NATURAL LANGUAGE SYNTHESIS

Explanation

복잡한 로직을
자연어로 풀이

ESCAPE VELOCITY

노가다, 단순 반복, 구문 오류라는 '중력'에서 탈출

PRODUCTIVITY BOOST

Antigravity 개념

- AI 기반 코드 생성 및 수정 도구
- 코드 설명, 리팩토링, 테스트 생성 지원
- 대화형 인터페이스 기반 개발 지원

코드를 대신 작성하는 도구가 아니라, 사고를 확장하는 도구임

스마트 에디터 vs 로봇 사원

구분	Cursor AI	Antigravity
정체성	AI 탑재 편집기	**일을 대신하는 로봇 사원**
작동 단위	코드 라인 / 함수	기능(Feature) / 태스크
권한	텍스트 편집 중심	파일 생성 + 터미널 + 브라우징
사용자 역할	운전자 (운전 보조)	**감독관 (목적지 설정 및 승인)**

스스로 생각하고, 행동하고, 수정하다

1 **자율 계획 (Autonomous Planning)**
작업 리스트 사전 설계

2 **도구 활용 (Tool Use)**
터미널 명령어 실행 및 패키지 설치

3 **자기 수정 (Self-Correction)**
에러 로그 분석 후 자체 코드 수정

Antigravity

비행 준비: 설치 가이드

Download

antigravity.google 접속
및 OS별 설치

Setup

기존 VS Code 설정
(테마, 단축키) 가져오기

Login

구글 계정으로
Gemini 3 모델 활성화

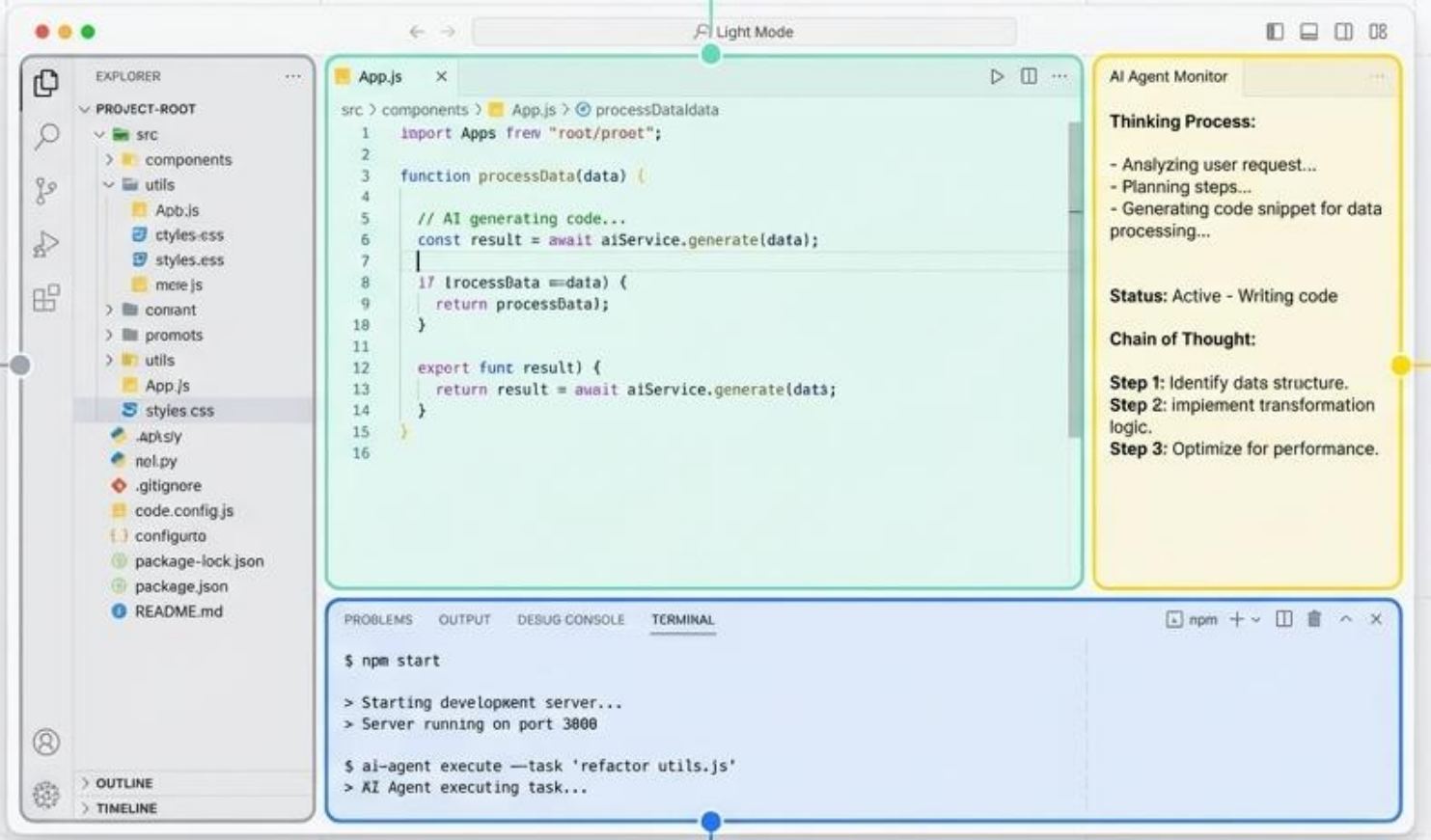
Extension

Chrome 확장 프로그램
설치

완벽히 통제 가능한 작업실

Center (Editor):
AI의 실시간 코드 타이핑 및 수정 영역

Left (File Tree):
기존과 동일한 프로젝트 구조 (친숙함)



Right (Agent Monitor):
[★ 핵심] AI의 사고 과정(Chain of Thought) 및 작업 상태 실시간 출력

Bottom (Terminal):
AI의 직접 명령어 입력 및 로그 확인 공간

기본 프롬프트 예시 – Python

정수 리스트를 입력받아 평균과 분산을 계산하는 함수를 작성해줘. Python으로 작성하고 예외 처리도 포함해줘.

```python
def calculate_stats(numbers):
    try:
        if not numbers:
            return None
        mean = sum(numbers) / len(numbers)
        variance = sum((x - mean) ** 2 for x
in numbers) / len(numbers)
        return mean, variance
    except TypeError:
        return "Input must be a list of
numbers"
```

Note
예외 처리 구조 확인

기본 프롬프트 예시 – C

문자열을 입력받아 역순으로 출력하는 함수를 C로 작성해줘. 동적 메모리 사용 버전으로.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char* reverse_string(const char* input) {
    int len = strlen(input);
    char* reversed = (char*)malloc(len + 1); // Allocation
    if (reversed == NULL) return NULL;

    // ... reversal logic ...

    return reversed;
}
```

malloc 사용 확인

NULL 포인터 체크

Antigravity

Mission 1: C와 Python의 가교

기존 C 코드를 Python으로 변환하는 첫 단계

Problem & Solution

Legacy Code (C)

```
int main() {
  int scores[] = {85, 92, 78, 95};
  int max = scores[0];

  for (int i = 1; i < 4; i++) {
    if (scores[i] > max) max = scores[i];
  }
}
```

Ctrl + I (Chat Window)

위 C 코드를 Python으로 변환해줘. 리스트 내장 함수를 사용하지 말고 우선 for문을 사용해서 작성해줘.

Ctrl + I로 코드 생성...

Mission 1: 최적화 (Optimization)

Prompt Request: "이번에는 Python의 내장 함수를 사용하여 가장 효율적인 코드로 수정해줘."

Step 1: Loop 방식

```
def find_max(scores):
    max_val = scores[0]
    for s in scores:
        if s > max_val:
            max_val = s
    return max_val
```

Step 2: Pythonic 방식

```
def find_max(scores):
    return max(scores)
```

단순 변환을 넘어 언어별 특성에 맞는 최적화 수행

Critical Thinking: 검증 및 디버깅

Scenario: 리스트가 비어있다면?

만약 scores 리스트가 비어있다면 max() 함수나 이전 코드는 에러가 발생하지 않니? 예외 처리를 포함해서 안전한 함수 형태로 다시 짜줘.

Flight Safety Check

- **정확성 (Accuracy)**
  C(sizeof) vs Python(len) 구분 확인
- **안전성 (Safety)**
  Null/Empty 데이터 방어
- **효율성 (Efficiency)**
  Pythonic한 코드 여부

Mission 2: AI의 비행 계획 읽기

Task

"C언어로 팩토리얼 계산기를 만들어줘"

→

Artifact (Manager View)

Implementation Plan (구현 계획) 확인

→

Review

"음수 입력 예외 처리가 계획에 있는가?" -> 수정 요청

동료를 100% 활용하는 법

90% 단축

생산성 임팩트:
보일러플레이트(기본 설정)
작성 시간 90% 단축

❌ **[Bad]** 로그인 만들어줘. (범위 과다)

✅ **[Good]** Node.js 환경에서 JWT 로그인 추가. DB는 SQLite, 비밀번호는 bcrypt 암호화.

Tip: 배경지식(Context)과 제약사항(Constraint)이 구체적일수록 퍼포먼스 상승

AI가 다 해주면 우리는 무엇을 하나요?

단순 코딩의 종말 ≠ 공부의 종말 (기초가 더 중요해지는 시대)

새로운 핵심 역량:

- AI 생성 코드의 보안 취약점 도출
- 비효율적 알고리즘 인지 및 개선 지시

지식의 역할 변화: '암기'하는 개발자 → '검수'하고 '설계'하는 엔지니어